QuickDraw Doodle Classification: Classical ML vs CNNs — Project Report

Oct 30, 2026

Team

Name	SRN
Vipin Lokesh	PES1UG24AM321
Samyuktha Vadrevu	PES1UG24AM312

Course: Machine Learning mini project. Repository: Vocod3r/ML_MiniProject_AM321_AM312.

The code splits the work into two workstreams: "Person A" (SamyukthaVadrevu)(dataset preparation and classical models) and "Person B" (Vipin V Lokesh)(CNN models, testing and the demo app).

Abstract

A small CNN beats every classical model on Google QuickDraw doodles: 96.37% test accuracy on 3 classes and 86.34% on 10 classes, against 95.18% and 82.63% for the best SVMs. The project compares Logistic Regression, a Linear SVM and SVMs with RBF, polynomial and sigmoid kernels against four progressively simplified CNNs (V1 to V4), all trained on the same 6,000 balanced images. It tests the classical models on 3, 10 and 50 classes, compares grayscale and binarized input for the CNNs, and analyses which doodles get confused. A Flask "Sketch Search" app lets a user draw a doodle, classifies it with the CNN and retrieves the most similar test images.

Introduction and objectives

QuickDraw is a set of millions of 28 x 28 hand-drawn sketches made in under 20 seconds each, so the same object is drawn in very different ways. The project replicates the setup of a reference paper (its Figures 2, 3, 8 and 9 are followed for the CNN versions, confusion matrices and training curves) and asks four questions:

1. How do linear and kernel classical models compare with CNNs on the same training budget?
2. How fast does accuracy fall as the number of classes grows from 3 to 10 to 50?
3. Does removing CNN layers (V1 to V4) or binarizing the input hurt accuracy, and by how much?
4. Which classes are confused with each other, and why?

Dataset and preprocessing

Three datasets were built from the public QuickDraw `numpy_bitmap` files, each with 20,000 images per class and a stratified 80/10/10 split (seed 42).

Dataset	Classes	Train / Val / Test	Script
3-class	banana, apple, cat	48,000 / 6,000 / 6,000	`src/prepare_data.py`
10-class	the 3 above + dog, car, bicycle, airplane, chair, tree, fish	160,000 / 20,000 / 20,000	`src/prepare_data_10.py`
50-class	airplane, alarm clock, ambulance … fish, flower (50 everyday objects and animals)	800,000 / 100,000 / 100,000	`src/prepare_data_50.py`

Each file (`dataset_Kclasses.npz`) stores `X_train` , `X_val` , `X_test` as float32 vectors of 784 pixels scaled to 0–1, integer labels, and the class names.

- **Classical models** use the flattened 784-pixel vectors directly.
- **CNNs** reshape each image to 28 x 28 x 1. Grayscale input is scaled to [0, 1]; binarized input sets every pixel above 127 to 1 and the rest to 0. The paper gives no threshold, so 127 is the team's choice.
- **Fair training budget:** every model in the final comparison trains on 6,000 balanced images (2,000 per class for 3 classes, 600 for 10, 120 for 50). Validation and test sets are never subsampled.

Methodology

Classical models (scikit-learn)

Model	Settings
Logistic Regression	lbfgs solver, max_iter 300
Linear SVM	`LinearSVC`, C = 1.0, max_iter 3000
RBF SVM	`SVC(kernel='rbf')`, C = 1.0, gamma = 'scale'
Polynomial SVM	degree 3, C = 1.0, gamma = 'scale'
Sigmoid SVM	C = 1.0, gamma = 'scale'

C for the RBF SVM was tuned on the 3-class validation set (`src/rbf_tuning.py`): C = 0.1 gave 94.45%, C = 1 gave 95.68%, C = 10 gave 95.55%, so C = 1 was kept for all kernels.

CNN versions (TensorFlow / Keras, `cnn_models.py`)

The four versions follow the paper's progressive simplification. The paper gives no filter counts or kernel sizes, so the team used 3 x 3 kernels, 32 then 64 filters, ReLU and 'same' padding.

Version	Layers	Parameters (3 / 10 classes)
V1	Conv32 → MaxPool → Conv64 → MaxPool → Dense 128 → Softmax	420,739 / 421,642
V2	Conv32 → MaxPool → Dense 128 → Softmax	803,651 / 804,554
V3	Conv32 → Dense 128 → Softmax	3,212,099 / 3,213,002
V4	Conv32 → Dense 64 → Softmax	1,606,211 / 1,606,666

Removing pooling layers makes the flattened layer larger, so the "simpler" V2 and V3 have more parameters than V1. V1, the smallest model, is also the most accurate.

All CNNs train with Adam (learning rate 0.001), sparse categorical cross-entropy, batch size 128 and a fixed 15 epochs with no early stopping, so training times are comparable.

Experimental setup

The first 3-class runs were not a fair comparison: Logistic Regression and the Linear SVM trained on all 48,000 images (93.17% and 93.25% test accuracy), while the kernel SVMs used only 6,000 because kernel SVM training grows roughly quadratically with sample count. Retrained on the same 6,000 images, the two linear models fall to 89.87% and 89.37%. Every result below therefore uses the 6,000-image budget.

1. Classical models: `src/fair_comparison.py`, `fair_comparison_10.py` and `fair_comparison_50.py` train all five models on the balanced 6,000-image subset for 3, 10 and 50 classes.
2. CNNs: `run_sweep.py` runs `train_cnn.py` for V1–V4 on grayscale and binarized input with `--train-samples 6000`, on 3 and 10 classes (16 runs).
3. Each run logs validation accuracy, test accuracy, training time and sample count in the same CSV columns; `combine_results.py` merges both workstreams into `all_model_results.csv`.
4. Each CNN run also saves a training curve and a normalized validation confusion matrix (`results/*_curve.png`, `*_confusion.png`).

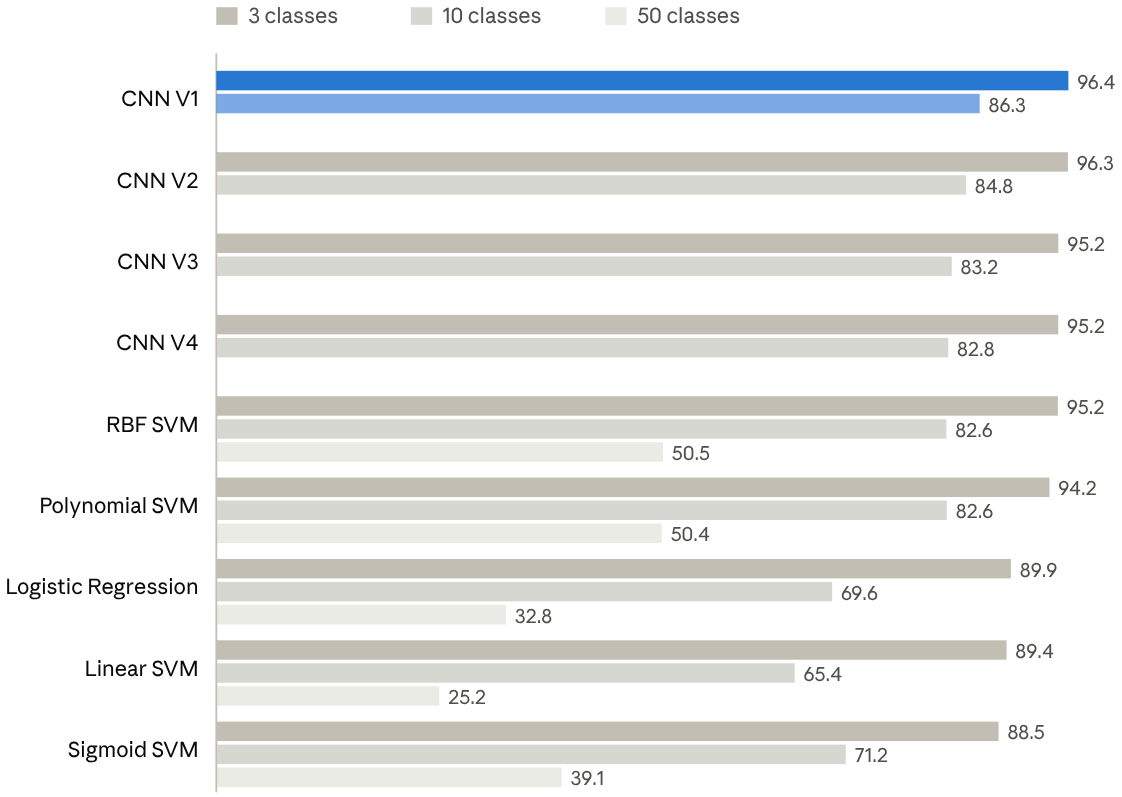
Training times are wall-clock seconds on the team's own machines and are only comparable within one workstream.

Results

CNN V1 (grayscale) is the best model at every class count it was run on: 96.37% on 3 classes and 86.34% on 10. The RBF and polynomial SVMs come within about 1 point on 3 classes but trail by about 4 points on 10. All classical models collapse at 50 classes, where the best (RBF SVM) reaches only 50.51%.

CNN V1 is the most accurate model; classical models lose 44 to 64 points from 3 to 50 classes

Test accuracy (%), all models trained on 6,000 balanced images



CNNs were run on 3 and 10 classes only; CNN rows use grayscale input.

Repository results/all_model_results.csv · test accuracy, 6,000 training images

CNN: binarized versus grayscale input

Binarizing the input costs every CNN version a little accuracy, and the cost grows with more classes. Grayscale keeps stroke-edge information that a hard threshold throws away.

Version	Test change, 3 classes (pts)	Test change, 10 classes (pts)	Training time, grayscale, 3 / 10 classes (s)
V1	−0.05	−0.89	31.2 / 58.1
V2	−0.52	−0.55	19.8 / 35.7
V3	−0.06	−1.22	33.1 / 57.7
V4	−0.22	−1.26	29.4 / 33.8

V2 is the fastest CNN and only 0.05 points behind V1 on 3 classes, but 1.55 points behind on 10 classes. The second convolution block matters more as the task gets harder.

Classical models: training time (s, 6,000 images)

Model	3 classes	10 classes	50 classes
Logistic Regression	0.97	3.67	9.73
Linear SVM	1.87	5.16	28.82
Sigmoid SVM	3.97	7.40	14.79
RBF SVM	5.49	7.28	16.65
Polynomial SVM	6.97	9.83	25.34

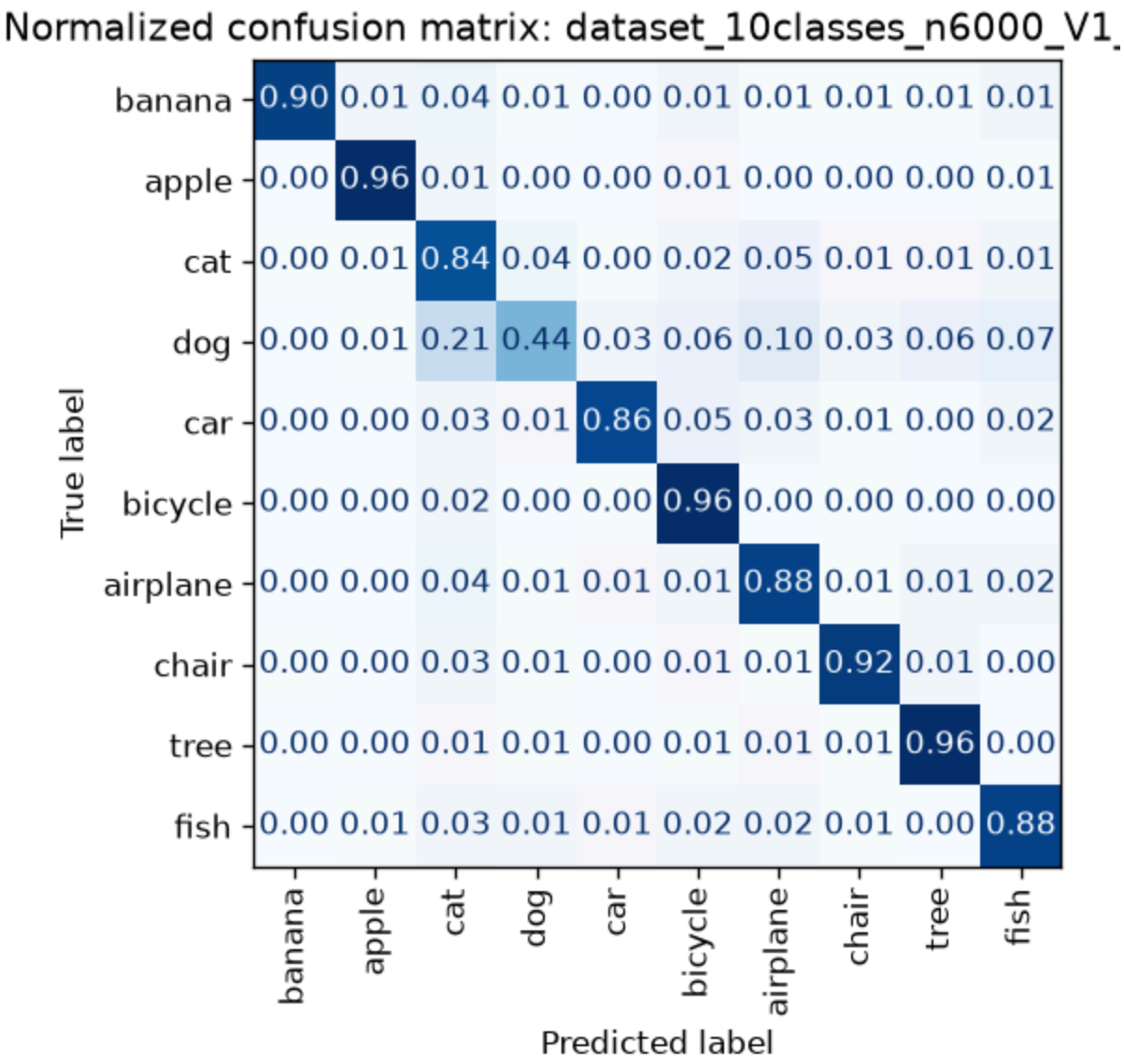
The classical models train in seconds, while each CNN takes 20 to 60 seconds for 15 epochs. The RBF SVM gives the best accuracy per second of training among the classical models.

Error analysis

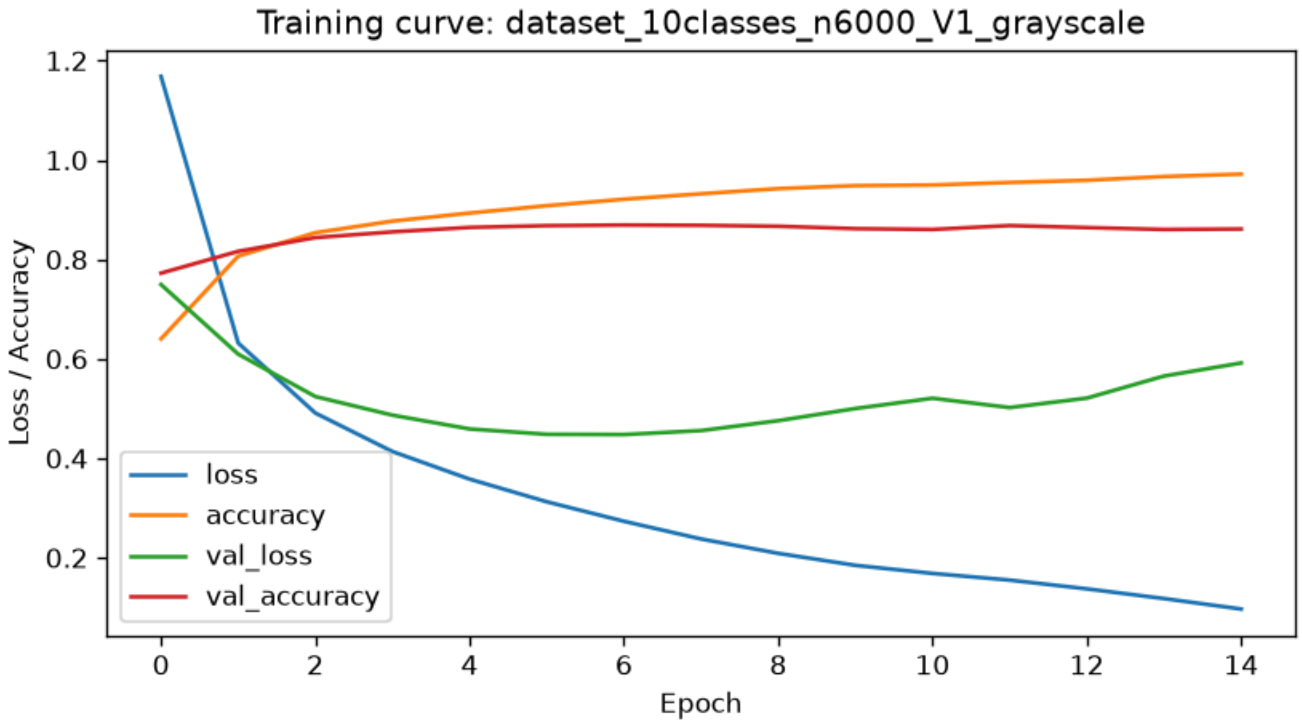
Most errors come from classes that share an outline: four-legged animals, rectangular objects and long thin shapes. Detail that tells them apart (ears, wheels, numbers) is small or missing at 28 x 28.

3 and 10 classes

- On 3 classes the RBF SVM's main error is banana drawn like a cat (118 of 2,000) and cat like banana (70); apple is almost never confused (96.1% recall).
- On 10 classes CNN V1 recognises apple, bicycle and tree at 96% but dog at only 44%. Dogs are labelled cat 21% of the time and airplane 10% of the time.



The CNN V1 training curve shows overfitting: training accuracy keeps rising to about 97% while validation accuracy flattens near 86% after epoch 4, and validation loss climbs again after epoch 6. Early stopping or dropout would likely help (see Discussion).

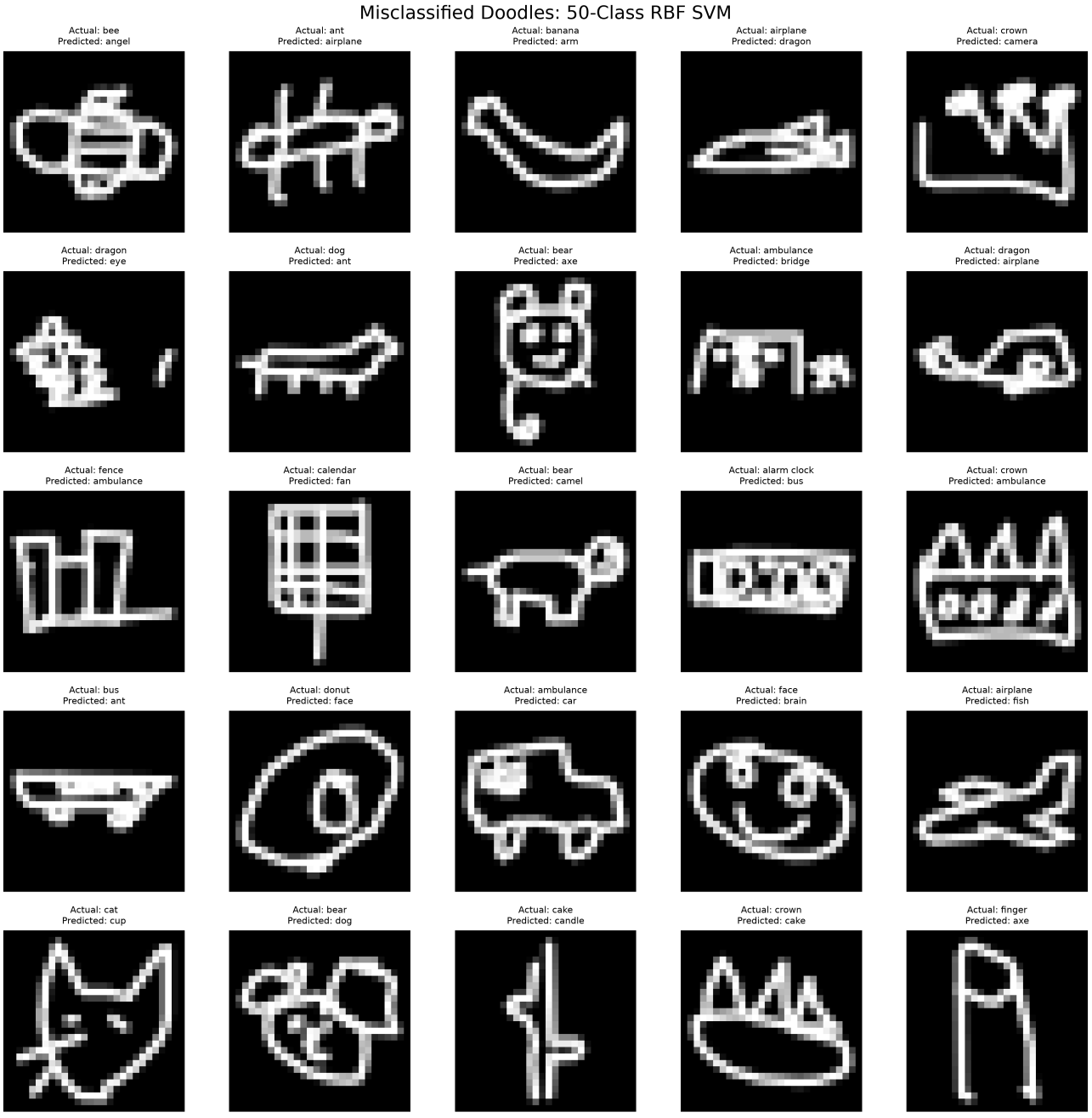


50 classes (RBF SVM, `src/error_analysis_50.py`)

The RBF SVM reaches 50.51% test accuracy with only 120 training images per class. F1 ranges from 0.83 (apple) and 0.81 (envelope) down to 0.14 (dog), 0.18 (dragon) and 0.20 (bear).

Actual class	Predicted as	Test images misclassified (of 2,000)
feather	carrot	318
arm	banana	283
dolphin	bridge	282
calendar	calculator	266
dog	cow	255
book	calendar	244
calculator	calendar	243
face	donut	236

Many pairs are confused in both directions (calendar and calculator, arm and banana, feather and carrot), which points to genuinely similar drawings rather than a biased classifier.



Sketch Search demo app

"Sketch Search" (`search_app.py` + `search_app.html`) is a Flask web app where a user draws a doodle, the 10-class CNN V1 names it, and the app shows the 12 most similar doodles from the 20,000-image test set.

1. **Index at start-up.** A second Keras model stops at the CNN's last hidden layer (Dense 128). Every test image is passed through it, giving a 128-number "fingerprint" that is L2-normalised.
2. **Redraw the sketch QuickDraw-style.** The browser sends pen strokes to `/api/search`. The server scales them into a 256 x 256 box with 12 px padding and a 14 px line width, centres them, and shrinks to 28 x 28 with Lanczos resampling, matching how the training data was made.
3. **Classify and fingerprint.** The CNN returns its top 3 guesses with probabilities, and the embedder returns the sketch's fingerprint.
4. **Search.** Cosine similarity against all stored fingerprints picks the 12 nearest doodles; a majority vote of their labels acts as a second opinion on the CNN's guess.

The app auto-detects a binarized model from its file name and applies the same preprocessing, and it refuses to start if the model's class count does not match the data file.

Testing and repository structure

`test_pipeline.py` holds 15 checks for the CNN code, built on synthetic "doodles" (lines, circles, squares and crosses with random position, size, thickness and noise) so the pipeline can be tested without real data. A working CNN must learn these shapes, which makes it an end-to-end check rather than a shape check. Its `--check` mode validates a dataset file before training. `test_everything.py` runs everything in 2–3 minutes: package checks, the 15 tests, every data file, a 1-epoch real training run, and the results merge.

Path	Purpose
`src/prepare_data*.py`, `make_subset.py`	Download QuickDraw classes and build the 3/10/50-class splits and 6,000-image subset
`src/logistic_regression.py`, `*_svm.py`, `rbf_tuning.py`	Individual classical models and C tuning
`src/fair_comparison*.py`	Same-budget comparison of all five classical models
`src/error_analysis*.py`, `plot_results.py`	Confusion matrices, confused pairs, misclassified examples, accuracy and time plots

Path	Purpose
`data.py`, `cnn_models.py`, `train_cnn.py`, `run_sweep.py`	CNN data loading, V1–V4 definitions, training and the full sweep
`combine_results.py`	Merges both workstreams into `all_model_results.csv`
`search_app.py`, `search_app.html`	Sketch Search demo
`test_pipeline.py`, `test_everything.py`	Tests
`results/`	All CSVs, text reports and figures

The dataset `.npz` files and trained `.keras` models are not committed (`data/*.npz` is git-ignored), so the prepare scripts and `run_sweep.py` must be run before the demo app.

Discussion and conclusion

Convolution pays off as the task grows: the gap between CNN V1 and the best SVM is 1.2 points on 3 classes and 3.7 points on 10. Kernel SVMs remain a strong, fast baseline, while linear models fall furthest because doodle classes are not linearly separable in raw pixel space.

- **Depth beats width.** V1 has the fewest parameters (about 421k) yet is the most accurate. Dropping pooling (V3) quadruples the dense layer to 3.2M parameters with lower accuracy, so pooling and the second convolution add useful translation tolerance.
- **Grayscale beats binarized**, by up to 1.26 points on 10 classes; anti-aliased edges carry information.
- **More classes hurt sharply.** With 120 training images per class, the best classical model gets about half of 50-class test images right.

Limitations

- CNNs were not run on the 50-class dataset, so the 50-class comparison covers classical models only.
- Results are from one seed (42) with no repeated runs, so differences under about 0.5 points (e.g. V1 vs V2 on 3 classes) may not be meaningful.

- The CNN filter sizes, counts and binarization threshold are the team's choices, since the paper does not state them.
- Each CNN trains for a fixed 15 epochs with no regularisation; the training curves show overfitting after about 5 epochs.
- Training times come from different machines for the two workstreams.

Future work

Run the CNN sweep on 50 classes; add dropout, data augmentation and early stopping; train on the full 16,000 images per class; and repeat runs over several seeds to report mean and spread.

Conclusion

On equal 6,000-image budgets, the two-block CNN (V1) with grayscale input is the most accurate model, reaching 96.37% on 3 classes and 86.34% on 10. The RBF SVM is the best classical alternative at a fraction of the training time, and the Sketch Search app shows the CNN's learned features also work for similarity search.